



The Manor Hotel CRM

Project Documentation & Setup Guide

Final Year Project - Business Information Technology
Case Study: The Manor Hotel, Nyarutarama, Kigali, Rwanda

Part I: System Documentation

This document explains **how the system actually works**: the technology stack and why each piece was chosen, the layered architecture, the database design, and - most importantly - the end-to-end *flow* of data for the operations that matter (signing in, enforcing roles, booking a stay, billing a guest). Read this alongside `docs/API.md` (endpoint reference), `docs/ARCHITECTURE.md` (high-level diagram + security layer) and `docs/PROJECT_STRUCTURE.md` (file layout).

1. What the system is

A CRM for a single hotel property ("The Manor Hotel") used internally by hotel staff - there is no public-facing guest portal. Four kinds of staff use it, each with a different slice of the system:

Role	Job in the hotel	What they use the CRM for
RECEPTION	Front desk	Guest profiles, rooms, reservations, billing, complaints
MARKETING	Marketing team	Campaigns, loyalty points
MANAGER	Operations manager	Everything reception + marketing can do
ADMIN	System administrator	Everything, plus creating/deactivating staff accounts

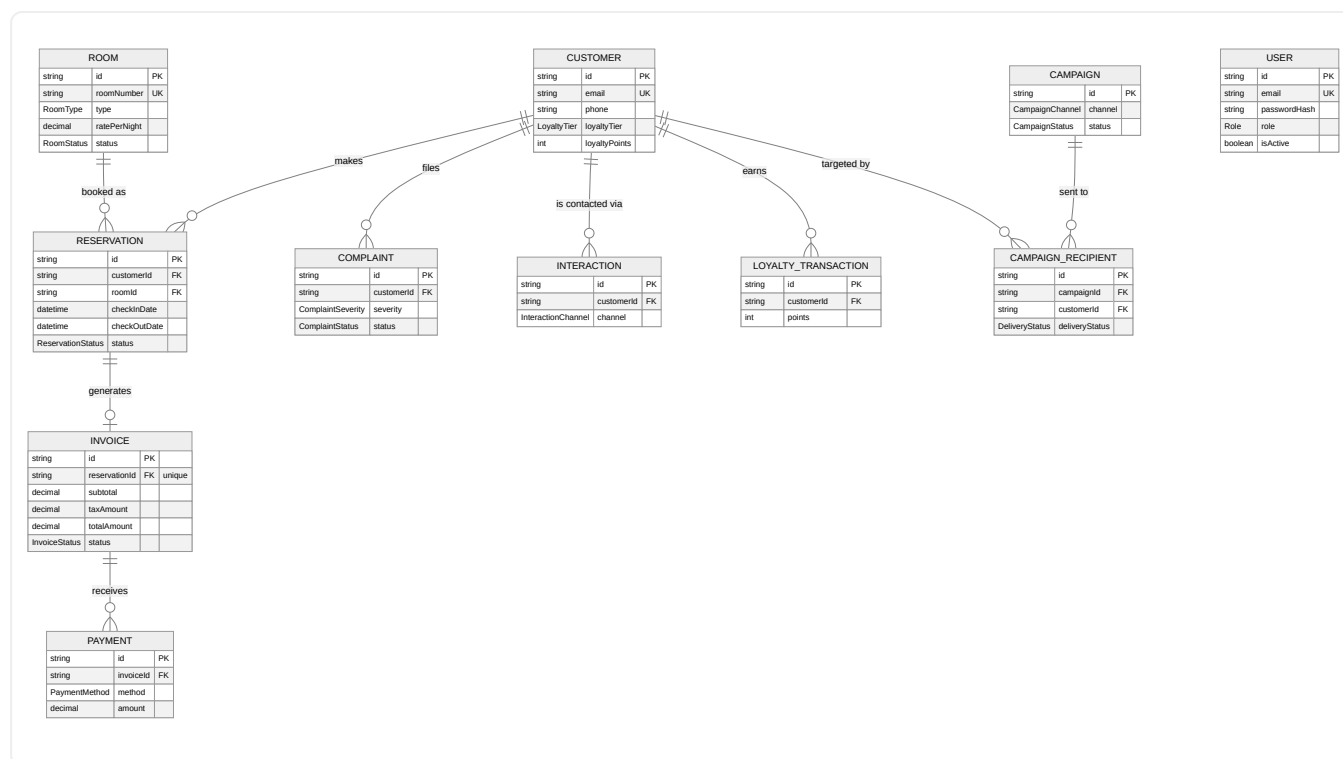
There is deliberately **no self-service sign-up** - see §4. An `ADMIN` provisions every account.

2. The stack, and why each piece is there

Layer	Technology	Why this one
Frontend	React + TypeScript + Vite	Vite gives near-instant dev reloads; TypeScript catches whole classes of bugs (wrong field names, wrong roles) before runtime - valuable when the same <code>Role</code> type is checked in dozens of places.
Routing	React Router	Nested routes map cleanly onto "authenticated vs not" and "this role vs not" gates (<code>ProtectedRoute</code> , <code>RoleRoute</code> - see §7).
Backend	Node.js + Express + TypeScript	Express is minimal and explicit - every middleware in the request pipeline is visible in <code>app.ts</code> (§6), which matters for explaining <i>exactly</i> how a request is authenticated and authorized, a core requirement for a CRM handling guest and payment data.
ORM	Prisma	Generates a fully-typed database client from <code>schema.prisma</code> (§3) - a typo'd column name or wrong type is a compile error, not a runtime surprise. <code>prisma migrate</code> also gives a reviewable, versioned history of every schema change (<code>backend/prisma/migrations/</code>).
Database	PostgreSQL	Free, open-source, industry-standard relational database - a correct fit for data that is inherently relational (a Reservation <i>belongs to</i> a Customer <i>and</i> a Room; an Invoice <i>belongs to</i> a Reservation).
Auth	JWT (jsonwebtoken) + bcrypt	Stateless tokens mean the API doesn't need a server-side session store; bcrypt is the standard for password hashing (§4).
Security middleware	helmet, express-rate-limit	helmet sets standard hardening HTTP headers; rate-limiting slows credential-stuffing against <code>/auth/login</code> .
Validation	zod	Every request body is parsed against a schema before touching the database - malformed input never reaches Prisma.
Testing	Vitest	Fast, TypeScript-native; used for the RBAC guard and error-mapping middleware (<code>backend/src/middleware/*.test.ts</code>), run in CI.
Containers	Docker + Docker Compose	One command reproduces the entire stack identically on any machine - see §8.
DB GUI	pgAdmin (containerized)	Lets anyone - a lecturer, a teammate - inspect real tables and rows without installing anything or knowing SQL.

Every item above is free and open-source - a hard requirement for this project.

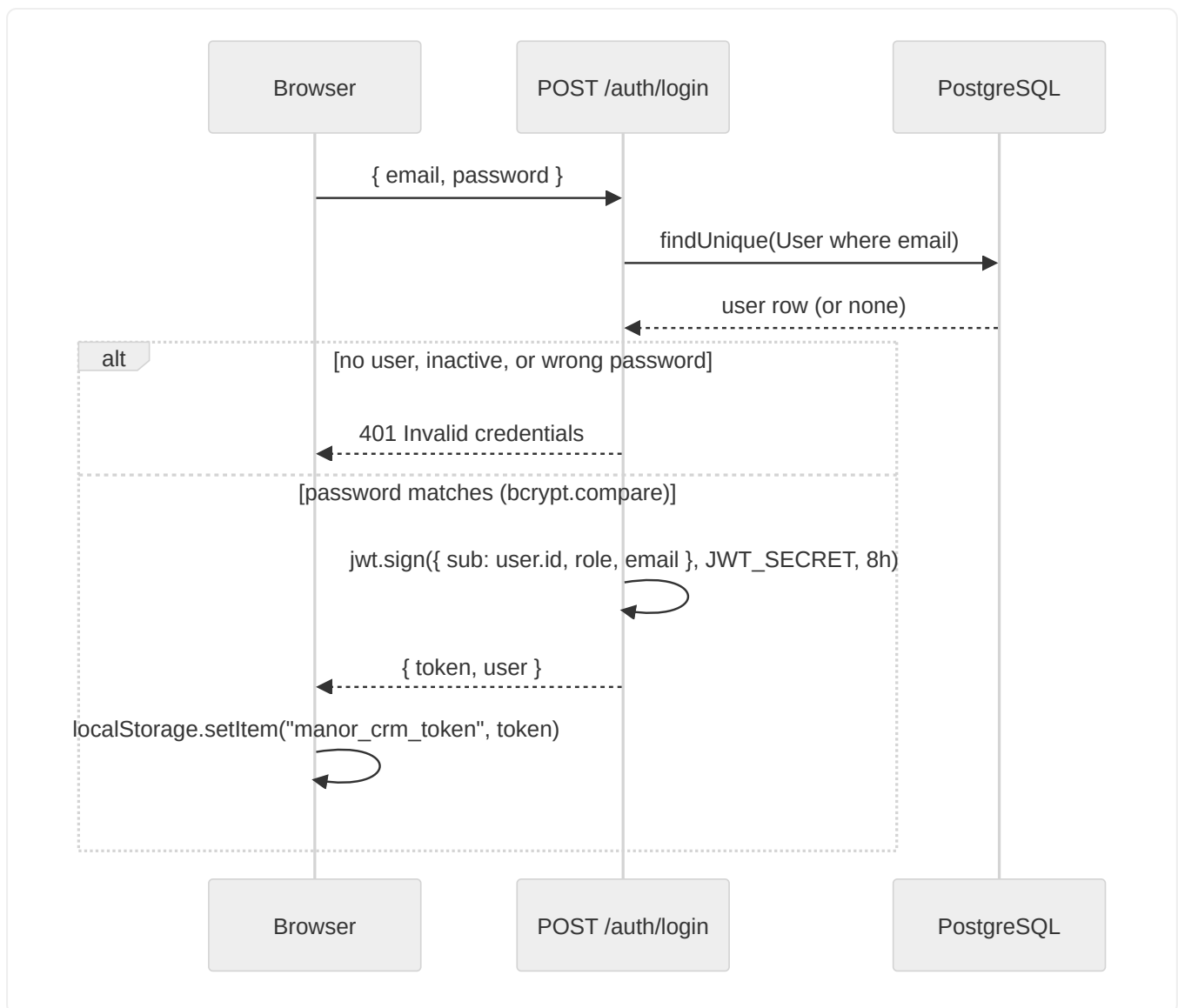
3. Database design



Notes on the design:

- User** is intentionally isolated - no foreign keys connect it to guest-facing tables. Staff identity and guest/business data are separate concerns; a staff account being deleted can never cascade into deleting reservations or invoices.
- Invoice.reservationId** is unique - a reservation has at most one invoice, generated on demand (§5.3), not automatically at booking time.
- Complaint.assignedToId** is a plain string, not a Prisma relation to **User** - a deliberate simplification (it's a label, not an enforced foreign key) rather than an oversight; a real next iteration would make it a proper relation.
- All monetary columns are **Decimal(10, 2)** (via `@db.Decimal`), never **Float** - avoids floating-point rounding errors in money math (see §5.3's tax calculation).

4. Authentication flow

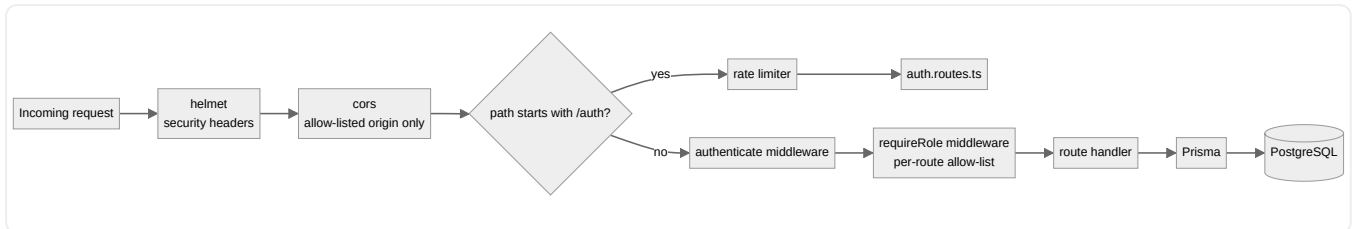


The frontend never stores a password - only the signed JWT. `AuthContext` decodes the token client-side (`atob` on the payload segment) purely to render the user's name/role in the UI; the decoded role is **never trusted for access control** - every protected action is re-checked server-side (§5), because a client-side check is trivially bypassable.

Why there's no `/auth/register`: earlier versions of this project had a public sign-up form that let anyone choose their own role - including `ADMIN`. That's a critical RBAC hole for a system that gates money (billing) and guest data behind roles. It was removed; see §5.4.

5. Authorization (RBAC) flow

Every request (other than `/health` and `POST /auth/login`) passes through the same middleware chain, wired up once in `backend/src/app.ts`:



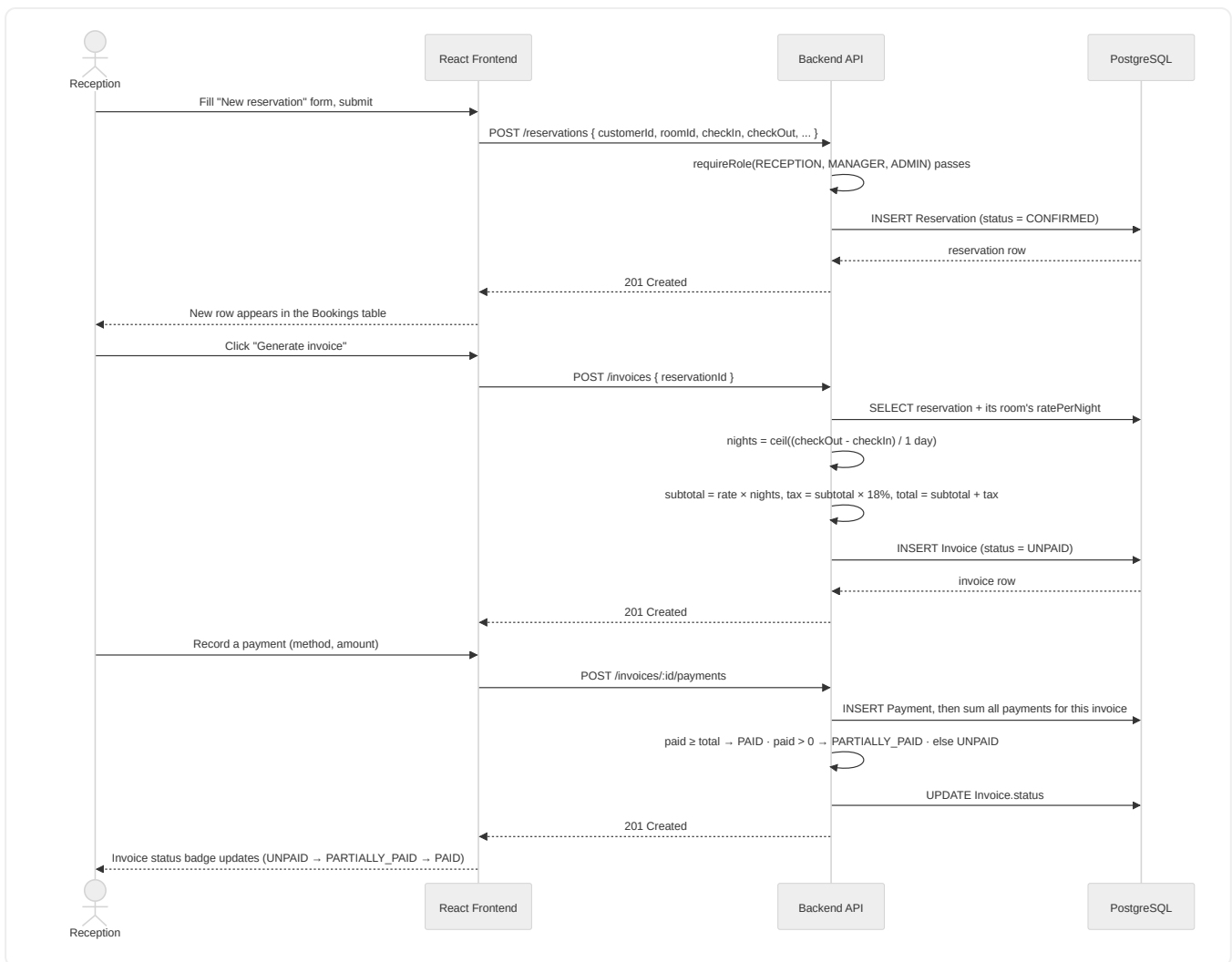
5.1 `authenticate` (`backend/src/middleware/auth.ts`)

1. Reads the `Authorization: Bearer <token>` header - missing header → `401`.
2. `jwt.verify(token, JWT_SECRET)` - invalid/expired signature → `401`.
3. **Re-fetches the user from the database by the token's `sub` claim** and checks `isActive`. This is the important part: it means deactivating a staff account (Staff page → Deactivate) revokes access **immediately**, on the very next request - not eight hours later when the token would otherwise expire. A JWT alone can't do this; the DB check is what makes it real.
4. Attaches `req.user = { sub, role, email }` for downstream middleware.

5.2 `requireRole(...roles)` (same file)

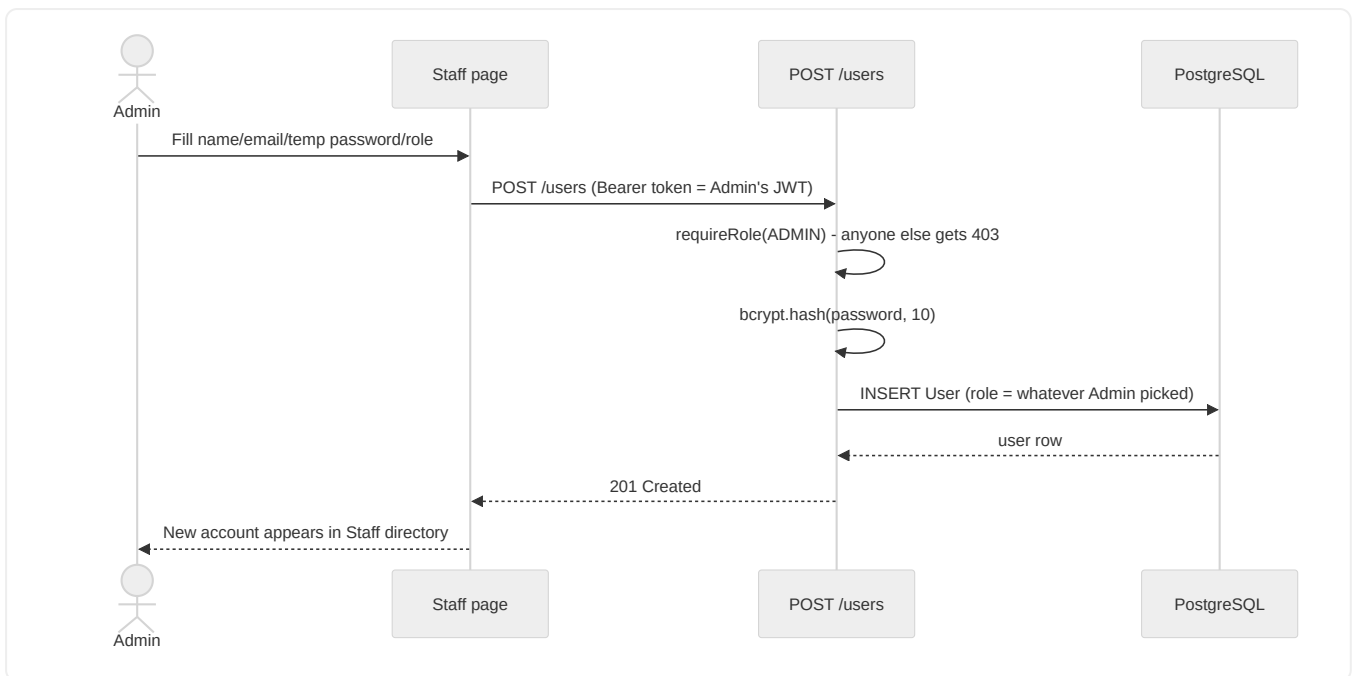
A tiny factory: `requireRole(Role.ADMIN)` returns middleware that checks `roles.includes(req.user.role)`, responding `403` if not. Every route file composes this per endpoint - e.g. `roomsRouter.post("/", requireRole(...MANAGEMENT), ...)`. The full matrix of who can do what is in the README's RBAC table and mirrors 1:1 with the `frontend/src/rbac.ts` constants used to hide nav links/routes the current user can't use anyway (a UX nicety - the backend guard is the actual security boundary, not the hidden nav link).

5.3 Worked example: booking a stay and billing it



This is a real, tested flow - not hypothetical: generating an invoice for a 3-night SUITE booking at \$180/night produces subtotal \$540.00, tax \$97.20, total \$637.20; a \$100 payment moves its status to **PARTIALLY_PAID**.

5.4 Staff provisioning (the RBAC fix, in detail)



Contrast this with the old, removed flow: a public `/auth/register` endpoint accepted a `role` field straight from an unauthenticated request body. Now, the **only** way a `role` field reaches the database is through this admin-gated endpoint - closing the hole completely rather than patching around it (e.g. by "hiding" the role picker in the UI while leaving the backend open, which would have been security theater).

6. Backend request pipeline (code map)

```
backend/src/
  app.ts           ← assembles the middleware chain above, mounts every router
  server.ts        ← just calls createApp().listen()
  rbac.ts          ← shared role-group constants (FRONT_DESK, MARKETING_TEAM, ...)
  lib/
    env.ts         ← validated environment variables (fails fast if missing)
    prisma.ts      ← the one PrismaClient instance, shared everywhere
    asyncHandler.ts ← wraps async route handlers so a thrown/rejected error
                      reaches errorHandler instead of hanging the request
  middleware/
    auth.ts        ← authenticate + requireRole (§5.1, §5.2)
    errorHandler.ts ← maps Prisma error codes to HTTP status (P2002 → 409,
                      P2025 → 404, P2003 → 400) instead of leaking stack traces
    rateLimit.ts   ← limits POST /auth/* to 20 requests / 15 min per IP
  routes/
    auth.routes.ts, users.routes.ts, customers.routes.ts, rooms.routes.ts,
    reservations.routes.ts, invoices.routes.ts, complaints.routes.ts,
    interactions.routes.ts, campaigns.routes.ts, loyalty.routes.ts,
    dashboard.routes.ts ← one file per resource; each declares its own
                          requireRole() calls per endpoint
```

Every route handler follows the same shape: `zod` schema validates the body → Prisma does the database work → response. Errors (bad input → `400` ; Prisma constraint violations → mapped by `errorHandler`) never crash the process - `asyncHandler` guarantees that.

7. Frontend architecture

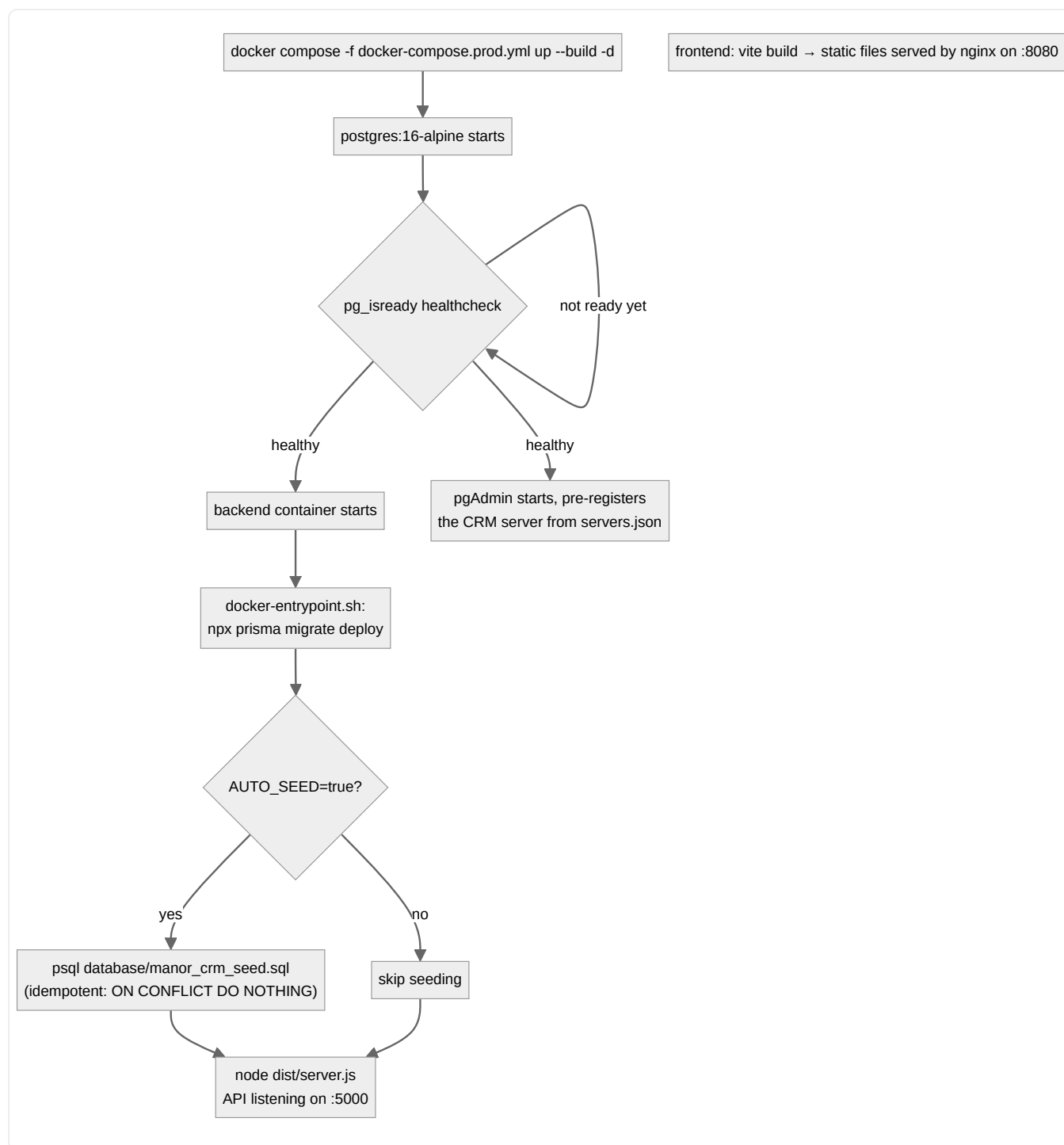
```
frontend/src/
  main.tsx → App.tsx      ← route tree
  auth/AuthContext.tsx    ← holds the JWT, decodes it client-side for display only,
                           exposes login()/logout(), attaches the token to every
                           outgoing request via api.ts
  api.ts                  ← thin fetch wrapper: adds Authorization header, throws
                           ApiError on non-2xx, calls the "unauthorized" handler
                           (which logs the user out) on any 401
  rbac.ts                 ← FRONT_DESK / MARKETING_TEAM / MANAGEMENT / ADMIN_ONLY
                           role-group arrays, mirroring the backend's rbac.ts
  components/
    ProtectedRoute.tsx    ← redirects to /login if not authenticated
    RoleRoute.tsx         ← redirects to / if the user's role isn't in the
                           allowed list for this route group
    Layout.tsx            ← sidebar + topbar; nav items filter themselves by role
  pages/                  ← one page per resource (Customers, Rooms, Reservations,
                           Invoices/Billing, Complaints, Campaigns, Loyalty, Staff)
```

Route tree shape (from `App.tsx`):

```
/login                                (public)
/ (Layout, requires auth)
/                                     Dashboard    (any role)
/customer                               Customers    (any role)
RoleRoute[FRONT_DESK]
  /rooms /reservations /invoices /complaints
RoleRoute[MARKETING_TEAM]
  /campaigns /loyalty
RoleRoute[ADMIN_ONLY]
  /staff
```

`RoleRoute` is a client-side convenience - it hides pages a role can't use so the UI doesn't show dead ends. It is **not** the security boundary; every one of those pages calls an endpoint that independently re-checks the role server-side (§5.2). Removing `RoleRoute` would make the UI show broken/forbidden pages, but would not open any actual security hole, because the backend doesn't trust the frontend.

8. Deployment flow (Docker)



`docker-compose.prod.yml` builds the backend from the **repo root** as its context (not just `backend/`) specifically so the Dockerfile can also `COPY database ./database` - the seed SQL needs to be inside the image for the entrypoint script to run it. The same image is used whether you're developing locally or demoing on a completely different machine, which is the point of containerizing in the first place: identical behavior everywhere.

See the root `README.md` for the actual commands and `pgAdmin` login details.

9. Security summary

Concern	Mitigation
Anyone self-assigning <code>ADMIN</code>	Public registration removed; only <code>ADMIN</code> can create accounts (§5.4)
Stolen/leaked JWT after account deactivation	<code>isActive</code> re-checked against the DB on every request, not just at login (§5.1)
Brute-forcing login	<code>express-rate-limit</code> on <code>/auth/*</code> (20 req / 15 min / IP)
Common web vulnerabilities (clickjacking, MIME sniffing, etc.)	<code>helmet</code> default header set
SQL injection	Prisma parameterizes all queries - no raw string-built SQL anywhere in the routes
Malformed request bodies reaching the DB	<code>zod</code> schema validation on every route before any Prisma call
Cross-origin requests from arbitrary sites	<code>cors</code> restricted to <code>CORS_ORIGIN</code> (the frontend's own origin)
Passwords at rest	<code>bcrypt</code> hashing (cost factor 10), never stored or logged in plaintext
Money rounding errors	<code>Decimal</code> columns + Prisma's <code>Decimal</code> type, not floating point

Part II: Setup, Running, and Deployment Guide

A full-stack hotel CRM built as a final-year Business Information Technology project: guest profiles, room and reservation management, billing/invoicing, complaint tracking, marketing campaigns, and a loyalty program, all behind JWT-authenticated, role-based access control (RBAC).

- Frontend: React + TypeScript + Vite (dashboard, customers, rooms, reservations, billing, complaints, campaigns, loyalty, staff administration), styled as a hotel/hospitality product
- Backend: Node.js + Express + TypeScript, modular routes/middleware, JWT auth with role-based route guards, helmet + rate limiting
- Database: PostgreSQL + Prisma ORM

Every tool in this stack is free and open-source (Node.js, PostgreSQL, Prisma, React, Express).

Project folders

- `backend/` Express API + Prisma schema (`src/routes` , `src/middleware` , `src/lib`)
- `frontend/` React UI
- `database/` SQL seed files for import
- `docs/` API, architecture, and defense-report documentation
- `postman/` ready API collection
- `scripts/` one-command setup scripts

Role-based access control (RBAC)

Roles: `ADMIN` , `MANAGER` , `MARKETING` , `RECEPTION` .

There is **no public self-registration**. Staff accounts are created only by an `ADMIN` from the in-app "Staff" page (or `POST /users`), which prevents anyone from granting themselves an elevated role - a gap in earlier versions of this project where the public sign-up form let a visitor pick their own role, including `ADMIN` .

Area	Read	Write
Customers	any authenticated staff	RECEPTION, MANAGER, ADMIN (delete: MANAGER, ADMIN)
Rooms	RECEPTION, MANAGER, ADMIN	MANAGER, ADMIN (status updates: also RECEPTION)
Reservations	RECEPTION, MANAGER, ADMIN	RECEPTION, MANAGER, ADMIN
Invoices & payments	RECEPTION, MANAGER, ADMIN	RECEPTION, MANAGER, ADMIN
Complaints	RECEPTION, MANAGER, ADMIN	RECEPTION, MANAGER, ADMIN
Interactions	any authenticated staff	any authenticated staff
Campaigns & loyalty	MARKETING, MANAGER, ADMIN	MARKETING, MANAGER, ADMIN
Staff accounts	ADMIN	ADMIN

Deactivating a staff account (`isActive = false`) takes effect immediately - the auth middleware re-checks account status on every request, not just at login/token-issue time.

Quick setup (recommended)

Windows PowerShell:

```
./scripts/setup.ps1
```

Linux/macOS:

```
chmod +x scripts/setup.sh
./scripts/setup.sh
```

Manual setup - local PostgreSQL (recommended on Linux)

1) Create the database and a dedicated role

```
sudo -u postgres psql \  
-c "CREATE USER manor_crm WITH PASSWORD 'manor_crm_dev_pw';" \  
-c "CREATE DATABASE manor_crm OWNER manor_crm;" \  
-c "GRANT ALL PRIVILEGES ON DATABASE manor_crm TO manor_crm;"
```

2) Start the backend

```
cd backend  
cp .env.example .env  
npm install  
npx prisma generate  
npx prisma migrate dev --name init  
npm run dev
```

Backend URL: `http://localhost:5000`

3) Start the frontend

Open a second terminal:

```
cd frontend  
cp .env.example .env  
npm install  
npm run dev
```

Frontend URL: `http://localhost:5173`

4) Import demo data (optional)

```
PGPASSWORD=manor_crm_dev_pw psql -h localhost -U manor_crm -d manor_crm -f  
database/manor_crm_seed.sql
```

Default test users in the seed file (all use password `Password123!`):

- `admin@manorhotel.com` - ADMIN
- `manager@manorhotel.com` - MANAGER

- `reception@manorhotel.com` - RECEPTION
- `marketing@manorhotel.com` - MARKETING

Sign in as `ADMIN` and use the **Staff** page to create any further accounts.

Fully Dockerized setup (recommended for demos / showing the database to your lecturer)

One command builds and runs the entire stack - PostgreSQL, backend, frontend, and a pgAdmin GUI - with migrations and demo data applied automatically on first boot:

```
docker compose -f docker-compose.prod.yml up --build -d
```

If your host's local PostgreSQL service is already running (see the manual setup above), it also listens on port 5432 and will conflict with the `db` container. Either stop it first (`sudo systemctl stop postgresql`) or this Docker stack, whichever you're not using right now.

Services once it's up:

- Frontend: `http://localhost:8080`
- Backend API: `http://localhost:5000`
- PostgreSQL: `localhost:5432` (user `postgres` , password `postgres` , db `manor_crm`)
- pgAdmin (database GUI): `http://localhost:8081`

Viewing the database tables in pgAdmin

This is the easiest way to show the actual database - tables, columns, rows - to a lecturer or during a defense, without needing the `psql` CLI:

1. Open `http://localhost:8081`.
2. Log in with email `lecturer@manorhotel.com` , password `manor_crm_dev_pw` (set in `docker-compose.prod.yml` ; change it there before a public demo if you want).
3. In the left tree, expand **Servers** - "Manor Hotel CRM (Docker)" is already registered.
4. Click it and enter the PostgreSQL password `postgres` when prompted (this one-time click is a pgAdmin security requirement; it isn't pre-filled on purpose).

5. Expand **Databases** → **manor_crm** → **Schemas** → **public** → **Tables** to see every table (**Customer** , **Reservation** , **Invoice** , **Payment** , **User** , etc.) - right-click any table → **View/Edit Data** → **All Rows** to show live rows.

Stop everything (add **-v** to also wipe the database volume for a clean slate):

```
docker compose -f docker-compose.prod.yml down
```

Alternative: Dockerized PostgreSQL only (no local install)

If you'd rather not touch your host PostgreSQL install but don't want the full stack above, this brings up just a database + its own pgAdmin (<http://localhost:8081> , same pre-registered server, on host port **5433** to avoid clashing with a local install):

```
docker compose up -d
```

Point **backend/.env** 's **DATABASE_URL** at

postgresql://postgres:postgres@localhost:5433/manor_crm?schema=public and continue from step 2 of the manual setup above.

Deploy to Vercel (free)

Vercel doesn't run a long-lived Express server or host a database directly, so this app deploys as **two separate Vercel projects** (frontend + backend), plus a free external Postgres. Every piece of this is free.

1) Create a free Postgres database (Neon)

1. Sign up at neon.tech (free tier, no credit card).
2. Create a project and database named **manor_crm** .
3. From the Neon dashboard, copy **two** connection strings:
 - The **pooled** connection string (has **-pooler** in the hostname) → this is **DATABASE_URL** .
 - The **direct** connection string (no **-pooler**) → this is **DIRECT_URL** .

2) Run migrations and seed against Neon (from your machine)

```
cd backend
DATABASE_URL="<neon direct connection string>" npx prisma migrate deploy
PGPASSWORD="<neon password>" psql "<neon direct connection string>" -f
../database/manor_crm_seed.sql
```

3) Deploy the backend

1. Push this repo to GitHub (already done if you're reading this from GitHub).
2. On vercel.com, sign up free (GitHub login is fastest) and **Add New Project** → import this repo.
3. Set **Root Directory** to `backend`.
4. Add environment variables: `DATABASE_URL` (pooled), `DIRECT_URL` (direct), `JWT_SECRET` (generate a real random value, e.g. `openssl rand -base64 32`), `CORS_ORIGIN` (leave as a placeholder for now, e.g. `http://localhost:5173` - you'll update it in step 5).
5. Deploy. Note the resulting URL, e.g. `https://manor-crm-backend.vercel.app`.

4) Deploy the frontend

1. **Add New Project** again, same repo, **Root Directory** set to `frontend`.
2. Add environment variable `VITE_API_BASE_URL` = the backend URL from step 3 (no trailing slash).
3. Deploy. Note the resulting URL, e.g. `https://manor-crm.vercel.app`.

5) Point the backend's CORS at the real frontend URL

Go back to the **backend** Vercel project → Settings → Environment Variables → update `CORS_ORIGIN` to the frontend URL from step 4, then redeploy the backend (Deployments → [...] → Redeploy) so the new value takes effect.

Sign in with the seeded accounts (password `Password123!`) once both are live.

Running backend tests

```
cd backend
npm test
```

Frontend pages

- **Login** - JWT sign-in (no public sign-up; accounts are admin-provisioned)
- **Dashboard** - live KPIs (customers, reservations, open complaints, revenue collected)
- **Customers** - guest directory with loyalty tier and points
- **Rooms** - inventory, rates, status, and a one-click starter seed
- **Reservations** - booking form and status updates (confirmed → checked-in → checked-out)
- **Billing** - generate invoices from reservations and record payments
- **Complaints** - service recovery log with severity and status tracking
- **Campaigns** - marketing outreach by segment/channel (MARKETING, MANAGER, ADMIN)
- **Loyalty** - award or redeem guest loyalty points
- **Staff** - create/deactivate staff accounts and change roles (ADMIN only)

Included endpoints

Every route except `/health` and `POST /auth/login` requires an `Authorization: Bearer <token>` header obtained from `/auth/login`. Individual routes are further restricted by role - see the RBAC table above. In the included Postman collection, run the **Login** request first - it stores the token in a collection variable used automatically by every other request.

- `GET /health`
- `POST /auth/login`
- `GET/POST /users`, `PATCH /users/:id` (ADMIN)
- `GET/POST/PATCH/DELETE /customers`, `/customers/:id`
- `GET/POST/PATCH/DELETE /rooms`, `/rooms/:id`, `POST /rooms/quick-seed`
- `GET/POST /reservations`, `PATCH /reservations/:id/status`
- `GET/POST /invoices`, `POST /invoices/:id/payments`
- `GET/POST/PATCH /complaints`, `/complaints/:id`
- `GET/POST /interactions`
- `GET/POST /campaigns`, `PATCH /campaigns/:id/status`
- `GET/POST /loyalty/transactions`
- `GET /dashboard/kpis`

Core modules covered

- Customers and profiles
- Rooms and reservations

- Invoices and payments
- Complaints and service recovery
- Guest interactions log
- Campaigns and recipients
- Loyalty points and transactions
- Staff/user administration (RBAC)
- KPI dashboard API

GitHub shipping checklist

- Ensure `.env` files are not committed (already in `.gitignore`)
- Commit this folder structure as-is
- Push repository to GitHub
- Add screenshots in a new `docs/screenshots/` folder when ready
- CI runs automatically from `.github/workflows/ci.yml` (build + backend tests)